



Actividad:

Actividad 2: "Interoperabilidad y Extracción de Datos con API's REST"

Asignatura:

Programación aplicaciones de red

Alumno:

- **Buenrostro Avila Abiel Gustavo**
- **Ramírez Rendon Naomi Elena**
- **Gómez González Victor Andres**

Programa Educativo: **IdelO**

Presentado a:

PROF. Joyner Mazariegos Rabanales

1. Introducción

Una **API REST** (Representational State Transfer) es un conjunto de convenciones arquitectónicas que permite que sistemas heterogéneos intercambien información a través del protocolo HTTP usando formatos estándar como JSON o XML. Su relevancia para la **Inteligencia Organizacional** radica en que habilita la integración de fuentes de datos dispersas —CRMs, ERPs, plataformas en la nube, servicios de terceros— sin necesidad de conocer su implementación interna. Esto permite construir flujos de extracción, transformación y análisis de datos que alimentan dashboards y reportes de toma de decisiones.

En esta actividad se construye un cliente Python que consume el endpoint público

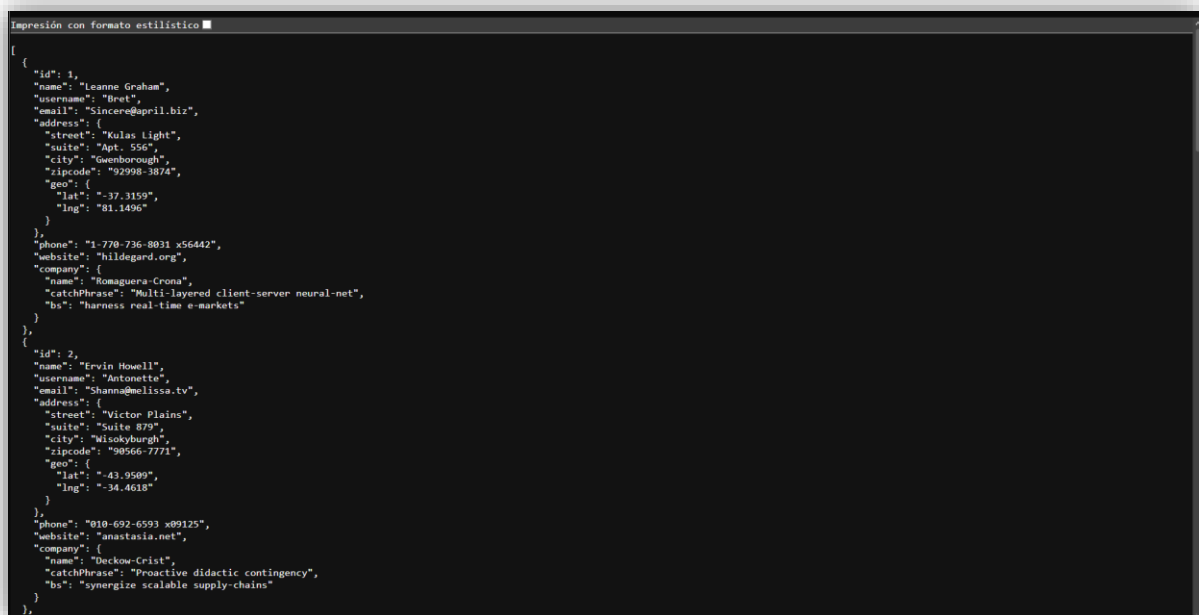
<https://jsonplaceholder.typicode.com/users>, extrae los datos del directorio de personal y genera un informe gerencial en consola.

2. Fase 1 — Preparación de la Abstracción de Red

API elegida:JSONPlaceholder (<https://jsonplaceholder.typicode.com>) Endpoint utilizado: **GET /users**

Justificación

Criterio	Detalle
Acceso	Gratuita, sin registro ni autenticación
Formato de respuesta	JSON limpio y bien estructurado
Contenido relevante	10 registros con campos name y company.name directamente aplicables
Estabilidad	Servicio de pruebas ampliamente utilizado en educación y desarrollo



```
Impresión con formato estilístico
{
  [
    {
      "id": 1,
      "name": "Leanne Graham",
      "username": "Bret",
      "email": "Sincere@april.biz",
      "address": {
        "street": "Kulas Light",
        "suite": "Apt. 556",
        "city": "Gwenborough",
        "zipcode": "92998-3874",
        "geo": {
          "lat": "-37.3159",
          "lng": "81.1496"
        }
      },
      "phone": "1-770-736-8031 x56442",
      "website": "hildegard.org",
      "company": {
        "name": "Romaguera-Crona",
        "catchPhrase": "Multi-layered client-server neural-net",
        "bs": "harness real-time e-markets"
      }
    },
    {
      "id": 2,
      "name": "Ervin Howell",
      "username": "Antonette",
      "email": "Shanna@melissa.tv",
      "address": {
        "street": "Victor Plains",
        "suite": "Suite 879",
        "city": "Missoula",
        "zipcode": "90566-7771",
        "geo": {
          "lat": "-43.9509",
          "lng": "-34.4618"
        }
      },
      "phone": "010-692-6593 x09125",
      "website": "anastasia.net",
      "company": {
        "name": "Deckow-Crist",
        "catchPhrase": "Proactive didactic contingency",
        "bs": "synergize scalable supply-chains"
      }
    }
  ]
}
```

3. Fase 2 — Construcción del Cliente REST

Flujo de la petición

```
Cliente Python
|
| HTTP GET https://jsonplaceholder.typicode.com/users
▼
Servidor JSONPlaceholder
|
| 200 OK + body (bytes UTF-8 con JSON)
▼
urllib.request.urlopen() → objeto HTTPResponse
|
| .read()
▼
bytes → .decode("utf-8") → str JSON
|
| json.loads()
▼
list[dict] (estructura nativa Python)
|
| filtrado: conservar name y company.name
▼
Informe gerencial en consola
```

Fragmento del código comentado

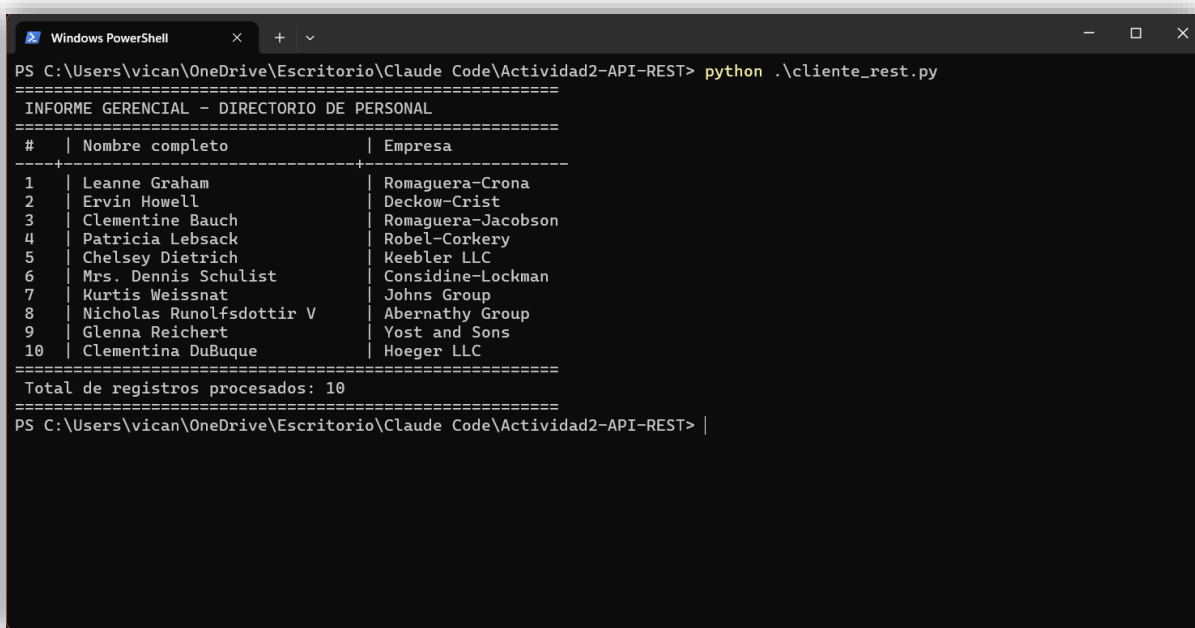
```
import urllib.request
import urllib.error
import json

URL_API = "https://jsonplaceholder.typicode.com/users"

def obtener_usuarios(url: str) -> list:
    # --- Fase 1: Petición HTTP GET ---
    try:
        with urllib.request.urlopen(url) as respuesta:
            if respuesta.status != 200:
                raise RuntimeError(f"Error HTTP: {respuesta.status}")
            # --- Fase 2: Decodificación JSON ---
```

```
datos_bytes = respuesta.read()
datos_json = json.loads(datos_bytes.decode("utf-8"))
return datos_json
except urllib.error.URLError as e:
    raise RuntimeError(f"No se pudo conectar con la API: {e.reason}") from e
```

El módulo `urllib.request` forma parte de la librería estándar de Python y proporciona una abstracción de alto nivel sobre los sockets TCP/IP. `urlopen()` gestiona internamente la resolución DNS, el establecimiento de la conexión TCP y el envío de la solicitud HTTP, devolviendo un objeto similar a un archivo del que se leen los bytes de la respuesta.



```
PS C:\Users\vican\OneDrive\Escritorio\Claude Code\Actividad2-API-REST> python .\cliente_rest.py
=====
INFORME GERENCIAL - DIRECTORIO DE PERSONAL
=====
# | Nombre completo | Empresa
-----+-----+-----
1 | Leanne Graham   | Romaguera-Crona
2 | Ervin Howell    | Deckow-Crist
3 | Clementine Bauch| Romaguera-Jacobson
4 | Patricia Lebsack| Robel-Corkery
5 | Chelsey Dietrich| Keebler LLC
6 | Mrs. Dennis Schulist| Considine-Lockman
7 | Kurtis Weissnat | Johns Group
8 | Nicholas Runolfsdottir V| Abernathy Group
9 | Glenna Reichert | Yost and Sons
10| Clementina DuBuque| Hoeger LLC
=====
Total de registros procesados: 10
=====
PS C:\Users\vican\OneDrive\Escritorio\Claude Code\Actividad2-API-REST> |
```

4. Fase 3 — Procesamiento e Inteligencia Organizacional

Filtrado de campos

La respuesta del endpoint `/users` devuelve para cada registro los siguientes campos:

Campo	¿Conservado?	Motivo
<code>id</code>	No	Identificador interno sin valor gerencial
<code>name</code>	Si	Nombre completo del colaborador
<code>username</code>	No	Alias de sistema, no relevante para el informe
<code>email</code>	No	Dato de contacto fuera del alcance del reporte
<code>address</code>	No	Información de ubicación no solicitada
<code>phone</code>	No	Dato de contacto fuera del alcance del reporte
<code>website</code>	No	No relevante para el directorio gerencial
<code>company.name</code>	Si	Empresa a la que pertenece el colaborador
<code>company.catchPhrase</code>	No	Eslogan interno, no relevante
<code>company.bs</code>	No	Texto de negocio interno, no relevante

Este proceso de selección de atributos es fundamental en Inteligencia Organizacional: de un objeto con más de 15 campos anidados, el informe retiene únicamente los 2 que responden a la pregunta de negocio planteada.

5. Fase 4 — Reflexión Arquitectónica

Pregunta 1: El proveedor migra su base de datos a la nube. ¿El script seguiría funcionando?

Sí, el script seguiría funcionando sin modificación alguna. La razón es el principio de **desacoplamiento** inherente a la arquitectura REST.

El cliente (nuestro script Python) solo conoce tres elementos del contrato de interfaz:

1. La **URL** del recurso: `https://jsonplaceholder.typicode.com/users`
2. El **método HTTP**: `GET`
3. El **formato de la respuesta**: JSON con la estructura acordada

La implementación del servidor —si usa MySQL en un VPS, Amazon DynamoDB, Google Firestore, o cualquier otro motor de base de datos— queda completamente oculta detrás de ese contrato. Este concepto se denomina **transparencia de ubicación** en arquitecturas distribuidas: el cliente no necesita saber dónde ni cómo se almacenan los datos, solo cómo pedirlos.

Los estándares HTTP son universales e independientes de la plataforma de alojamiento. Una migración a la nube que no altere la URL ni el esquema JSON de respuesta es, desde la perspectiva del cliente, completamente invisible. Este desacoplamiento es precisamente la fortaleza que hace a REST el estilo arquitectónico dominante en la integración de sistemas empresariales e Inteligencia Organizacional.

Pregunta 2: El proveedor migra de HTTP a HTTPS. ¿Qué implica eso?

La migración de `http://` a `https://` implica incorporar la capa **TLS/SSL** (Transport Layer Security) sobre el protocolo HTTP estándar. Esta transición aporta tres garantías fundamentales:

Garantía	Descripción
Confidencialidad	El contenido del mensaje (cabeceras y cuerpo) viaja cifrado; un atacante en red no puede leerlo
Integridad	TLS detecta cualquier modificación de los datos en tránsito mediante códigos de autenticación de mensaje (MAC)
Autenticación del servidor	El certificado digital firmado por una CA (Autoridad Certificadora) garantiza que el cliente habla con el servidor legítimo, no con un impostador

El riesgo que se mitiga con HTTPS es el ataque **Man-in-the-Middle (MITM)**: sin cifrado, un atacante posicionado entre el cliente y el servidor puede leer, modificar o inyectar respuestas. Con TLS, el handshake inicial (que usa criptografía asimétrica RSA/ECDSA para intercambiar una clave de sesión simétrica AES) hace este ataque computacionalmente inviable.

Desde el punto de vista del script, el cambio solo requiere actualizar la URL de `http://` a `https://`; `urllib.request` gestiona el handshake TLS automáticamente. JSONPlaceholder ya opera sobre HTTPS (puerto 443), por lo que la actividad ya sigue esta buena práctica.

6. Conclusiones

Conclusion — Buenrostro Avila Abiel Gustavo

A través de este proyecto, comprendí la importancia de la interoperabilidad mediante el uso de Python y la librería urllib, logrando abstraer la complejidad de las peticiones HTTP para conectar sistemas heterogéneos. Lo más relevante fue experimentar el proceso de transformar datos crudos en bytes hacia estructuras de datos nativas, lo que me permitió entender cómo se establece un canal de comunicación eficiente entre un cliente y un servidor. En el ámbito profesional, aplicaré estas bases para automatizar la extracción de información desde diversas fuentes, garantizando procesos de integración limpios y altamente portables.

Conclusion — Ramírez Rendon Naomi Elena

Mi aprendizaje principal se centró en la fase de procesamiento y filtrado de datos, donde identifiqué que la Inteligencia Organizacional depende de la capacidad de transformar grandes volúmenes de información en conocimiento útil. Al realizar el data wrangling sobre el JSON recibido, aprendí a priorizar únicamente los indicadores que responden a las necesidades del negocio, como el nombre del colaborador y su empresa. Esta experiencia me permitió valorar el rol del analista al reducir el ruido informativo, una habilidad que aplicaré para diseñar reportes ejecutivos que faciliten la toma de decisiones estratégica.

Conclusion - Gómez González Victor Andres

Esta actividad me permitió profundizar en la arquitectura REST y la importancia de la seguridad en la transferencia de datos. Entender conceptos como el desacoplamiento y la transparencia de ubicación me dejó claro que un sistema bien diseñado es resiliente ante cambios internos del proveedor, como migraciones a la nube. Asimismo, el análisis del protocolo HTTPS reafirmó la necesidad de implementar TLS para garantizar la integridad y confidencialidad de la información. Estos conocimientos técnicos me servirán para proponer soluciones de software que no solo sean funcionales, sino también robustas, escalables y seguras.

7. Repositorio en GitHub

URL del repositorio: [[URL del repositorio](#)]